SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools. *(BL4,BL5)*

Assessment Tool Weightage: 30%

15/20

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I Gk. Vyshnavi | 192425069 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

GVyshnavi

stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

1. Pseudocode to Calculate Network Throughput and Packet Loss

Algorithm: Network Performance Analysis

START

```
SET totalBitsReceived = 8000
SET timeTaken = 4
SET packetsSent = 200
SET packetsReceived = 180

IF timeTaken <= 0 THEN
   DISPLAY "Error: Time cannot be zero or negative."

ELSE IF packetsSent <= 0 THEN
   DISPLAY "Error: Total packets sent cannot be zero or negative."

ELSE

  CALCULATE throughput = totalBitsReceived / timeTaken

  CALCULATE packetsLost = packetsSent - packetsReceived

  CALCULATE packetLossPercentage =
        (packetsLost / packetsSent) * 100
```

```
DISPLAY "----- Network Performance Summary -----"
DISPLAY "Total Data Received: ", totalBitsReceived, " bits"
DISPLAY "Time Taken: ", timeTaken, " seconds"
DISPLAY "Packets Sent: ", packetsSent
DISPLAY "Packets Received: ", packetsReceived
DISPLAY "Packets Lost: ", packetsLost
DISPLAY "Network Throughput: ", throughput, " bps"
DISPLAY "Packet Loss Percentage: ",
    packetLossPercentage, "%"

IF throughput > 1500 AND packetLossPercentage < 10 THEN

  SET networkStatus = "Efficient"

ELSE

  SET networkStatus = "Inefficient"

END IF

DISPLAY "Network Classification: ", networkStatus
DISPLAY "---------------------------------------"

END IF
```

STOP

Explanation

The algorithm first stores the total number of bits received, time taken, packets sent, and packets received. It validates that the **time taken** and **packets sent** are not zero or negative to prevent division errors.

The network throughput is calculated using:

Throughput = Total Data Received / Time Taken

Therefore:

Throughput = 8000 / 4 = 2000 bps

Packet loss is first calculated as:

Packets Lost = Packets Sent − Packets Received

Packets Lost = 200 − 180 = 20 packets

The packet loss percentage is then calculated as:

Packet Loss Percentage = (Packets Lost / Packets Sent) × 100

Packet Loss Percentage = (20 / 200) × 100 = 10%

The algorithm checks whether the throughput is greater than **1500 bps** and packet loss is less than **10%**. Since the throughput is **2000 bps**, but the packet loss is exactly **10%** and not less than 10%, the network is classified as:

Network Status: Inefficient

The final output displays a complete summary of the calculated network performance metrics.

2. Pseudocode to Monitor Bandwidth Utilization and Detect Overloaded Links

Algorithm: Bandwidth Utilization Monitoring

```
START

  SET numberOfLinks = INPUT("Enter the number of network links")

  DECLARE bandwidthUtilization[numberOfLinks]

WHILE TRUE

  DISPLAY "----- Network Link Monitoring -----"

FOR i = 1 TO numberOfLinks

   DISPLAY "Enter bandwidth utilization percentage for Link ", i

   INPUT bandwidthUtilization[i]

   IF bandwidthUtilization[i] > 80 THEN

     DISPLAY "WARNING: Link ", i, " is overloaded."
```

```
        DISPLAY "Recommendation: Switch traffic to a backup link."
    ELSE
        DISPLAY "Link ", i, " is operating normally."
    END IF
  END FOR
  DISPLAY "All network links have been monitored."
  DISPLAY "Waiting before next monitoring cycle..."
  WAIT FOR 60 SECONDS
 END WHILE
STOP
```

Explanation

The algorithm is designed to continuously monitor the bandwidth utilization of multiple network links connecting different branches of an organization.

First, the administrator enters the **number of network links**. An array called **bandwidthUtilization** is created to store the utilization percentage of each link.

A **WHILE TRUE loop** is used to continuously monitor the network. Inside this loop, a **FOR loop** checks each network link one by one. The bandwidth utilization value of each link is collected and stored in the array.

The algorithm then checks whether the bandwidth utilization is greater than **80%**. If the utilization exceeds 80%, the link is considered **overloaded**. A warning message is displayed, and the algorithm recommends switching some or all traffic to a backup link.

If the utilization is 80% or below, the link is considered to be operating normally.

After checking all links, the algorithm waits for **60 seconds** before repeating the monitoring process. This allows the administrator to continuously track changes in bandwidth usage.

How the Algorithm Supports Efficient Bandwidth Management

This algorithm supports efficient bandwidth management by continuously monitoring the utilization of all network links. By detecting overloaded links when utilization exceeds **80%**, the network administrator can take action before the link becomes fully congested.

Redirecting traffic to a backup link helps distribute network traffic more evenly and prevents excessive congestion. The use of arrays makes it easier to store and manage utilization values for multiple links, while loops allow the monitoring process to continue automatically at regular intervals.

Therefore, the algorithm helps improve **network performance, bandwidth utilization, traffic management, and congestion prevention**.

3. Pseudocode to Select the Best Network Path Based on Link Quality

Algorithm: Dynamic Network Path Selection and Monitoring

```
START

  SET numberOfBranches = 6

  DECLARE bandwidth[numberOfBranches]
  DECLARE delay[numberOfBranches]
  DECLARE utilization[numberOfBranches]
  DECLARE linkStatus[numberOfBranches]
  DECLARE qualityScore[numberOfBranches]

  WHILE TRUE

    DISPLAY "----- Network Path Monitoring -----"

    FOR i = 1 TO numberOfBranches

      DISPLAY "Enter bandwidth for Branch ", i
      INPUT bandwidth[i]

      DISPLAY "Enter delay for Branch ", i
      INPUT delay[i]

      DISPLAY "Enter link utilization percentage for Branch ", i
```

```
FOR i = 2 TO numberOfBranches

  IF qualityScore[i] > highestScore THEN

    SET highestScore = qualityScore[i]

    SET bestPath = i

  END IF

END FOR


  DISPLAY "----- Network Performance Summary -----"

  FOR i = 1 TO numberOfBranches

    DISPLAY "Branch ", i
    DISPLAY "Bandwidth: ", bandwidth[i]
    DISPLAY "Delay: ", delay[i]
    DISPLAY "Utilization: ", utilization[i], "%"
    DISPLAY "Quality Score: ", qualityScore[i]

  END FOR

  DISPLAY "Best Communication Path: Branch ", bestPath
  DISPLAY "Highest Quality Score: ", highestScore

  DISPLAY "Monitoring will continue..."

  WAIT FOR 60 SECONDS

END WHILE
```

Explanation

The algorithm is designed to continuously monitor the network links connecting the headquarters to six branch offices. Since network conditions can change throughout the day, the algorithm repeatedly collects information about the **bandwidth, delay, utilization, and status** of each link.

Arrays are used to store the values for all six branches. A **WHILE TRUE loop** allows the monitoring process to continue continuously, while a **FOR loop** collects the network metrics for each branch during every monitoring cycle.

The algorithm calculates a **link quality score** using the following logic:

Quality Score = Bandwidth − Delay − Utilization

A higher bandwidth improves the quality score because it allows faster data transmission. Higher delay and higher utilization reduce the quality score because they can cause slower communication and network congestion.

If a link has failed, its quality score is set to **0**, and an alert is generated for the network administrator. The algorithm then ignores that failed link when selecting the best available path.

If a link has a utilization greater than **80%**, it is considered overloaded. A warning is displayed, and the system recommends switching traffic to an alternate path with a better quality score.

After calculating the quality scores, another loop compares all available paths and identifies the path with the **highest quality score**. This path is selected as the best communication path. The algorithm waits for **60 seconds** and then repeats the entire process. This allows it to adapt to changing bandwidth, delay, and utilization conditions.

How the Algorithm Improves Network Performance

This algorithm improves **network reliability** by continuously monitoring the status of every link and detecting failures quickly. When a link fails or becomes overloaded, an alternate path can be selected instead of continuing to send traffic through a poor-quality connection.

It improves **routing efficiency** because the path is selected based on multiple factors—bandwidth, delay, and utilization—rather than simply choosing the shortest path.

The algorithm also provides **fault tolerance** by identifying failed links and switching communication to another available path. Continuous monitoring allows the network to adapt to changing conditions and maintain stable communication between the headquarters and branch offices.

this approach helps provide a more **reliable, efficient, and adaptive network,** reducing congestion and improving overall communication performance.

RICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient